

Code Smell

Pedro Negri Leão Lambert

Mátricula: 819872

Disciplina: Teste de Software

24/05/2026

Este trabalho teve como objetivo analisar e refatorar um projeto JavaScript de geração de relatórios, buscando identificar Bad Smells no código de produção e melhorar sua clareza, manutenção e testabilidade. A versão original da classe **ReportGenerator** funcionava para os cenários cobertos pelos testes, mas concentrava muitas responsabilidades no método **generateReport**, além de possuir condicionais aninhadas, duplicação de lógica e valores fixos espalhados pelo código. A partir da análise manual e da execução do ESLint com o plugin SonarJS, foi criado o arquivo **src/ReportGenerator.refactored.js**, mantendo a suíte de testes como rede de segurança.

Link do repositório GitHub: <https://github.com/Pedro-nll/bad-smells-js-refactoring>

O primeiro Bad Smell identificado foi o método longo. O método **generateReport** acumulava a criação do cabeçalho, a filtragem dos itens, a montagem do corpo, a aplicação de regra de prioridade e a geração do rodapé. Esse acúmulo torna o código mais difícil de entender, pois uma alteração simples, como mudar apenas o formato HTML, exige a leitura de toda a lógica do método. Para os testes, esse problema também é relevante porque o comportamento final depende de vários caminhos internos misturados em uma única função.

O segundo Bad Smell encontrado foi a presença de condicionais aninhadas e complexidade cognitiva elevada. A classe precisava decidir ao mesmo tempo o tipo do relatório, o papel do usuário e as regras de visibilidade de cada item. Isso gerava muitos blocos **if/else** dentro de outros blocos, aumentando o risco de erro ao adicionar novos formatos ou novos papéis de usuário. Esse tipo de estrutura é problemático para manutenção porque cada nova regra tende a ampliar ainda mais a árvore de decisões.

O terceiro Bad Smell foi a duplicação de lógica. A soma do total e a montagem das linhas do relatório apareciam repetidas nos fluxos de CSV e HTML, tanto para administradores quanto para usuários comuns. Além disso, os limites de negócio, como o valor máximo de visualização do usuário comum e o valor que torna um item prioritário para administradores, estavam escritos diretamente no método. Esses números mágicos reduzem a expressividade do código e tornam mudanças futuras mais propensas a inconsistência.

Relatório da ferramenta

Antes da refatoração, o comando **npx eslint src/** apontou problemas no arquivo original. O resultado principal foi a complexidade cognitiva do método **generateReport**, que chegou a 27

enquanto o limite configurado era 5. A ferramenta também apontou um **if** que poderia ser simplificado por estar colapsado com outro condicional.

```
/src/ReportGenerator.js

11:3   error   Refactor this function to reduce its Cognitive Complexity from 27 to the 5
allowed sonarjs/cognitive-complexity

43:14  error   Merge this if statement with the nested one
sonarjs/no-collapsible-if

2 problems (2 errors, 0 warnings)
```

O `eslint-plugin-sonarjs` ajudou a quantificar um problema que a análise manual já sugeria: o método era difícil de acompanhar por causa do número de decisões internas. A análise manual permite perceber duplicações e responsabilidades misturadas, mas a ferramenta torna o diagnóstico mais objetivo ao indicar exatamente onde a complexidade ficou acima do limite configurado.

Processo de refatoração

O smell mais crítico escolhido para correção foi o método longo com alta complexidade cognitiva. Na versão original, a mesma função decidia o formato do relatório, percorria os itens, verificava o papel do usuário, aplicava regra de prioridade e calculava o total. Um trecho representativo da versão original mostra a mistura de decisões:

```
for (const item of items) {
  if (user.role === 'ADMIN') {
    if (item.value > 1000) {
      item.priority = true;
    }

    if (reportType === 'CSV') {
      report += `${item.id},${item.name},${item.value},${user.name}\n`;
      total += item.value;
    } else if (reportType === 'HTML') {
      const style = item.priority ? ' style="font-weight:bold;"' : '';
      report += `<tr${style}><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`;
      total += item.value;
    }
  }
}
```

```
}
```

Depois da refatoração, foi aplicada principalmente a técnica **Extract Method**. O método principal passou a apenas coordenar a geração do relatório, delegando a filtragem, o cálculo do total, a montagem do cabeçalho, do corpo e do rodapé para métodos menores e mais específicos. Também foram criadas constantes para substituir números mágicos e strings de papel/formato.

```
generateReport(reportType, user, items) {  
  const visibleItems = this.getVisibleItems(user, items);  
  const total = this.calculateTotal(visibleItems);  
  
  return [  
    this.buildHeader(reportType, user),  
    this.buildBody(reportType, user, visibleItems),  
    this.buildFooter(reportType, total),  
  ].join('').trim();  
}
```

A filtragem dos itens também foi isolada. Com isso, a regra de que administradores veem todos os itens e usuários comuns veem apenas itens de valor até 500 ficou explícita em um ponto único do código:

```
getVisibleItems(user, items) {  
  if (user.role === USER_ROLES.ADMIN) {  
    return items;  
  }  
  
  if (user.role === USER_ROLES.USER) {  
    return items.filter((item) => item.value <= STANDARD_USER_MAX_VALUE);  
  }  
  
  return [];  
}
```

Após essa mudança, o comando **npx eslint src/ReportGenerator.refactored.js** não apontou erros. A suíte completa também foi executada com sucesso, incluindo os testes originais e a cópia apontada para o arquivo refatorado.

```
PASS tests/ReportGenerator.test.js
```

```
PASS tests/ReportGenerator.refactored.test.js
```

```
Test Suites: 2 passed, 2 total
```

```
Tests:      10 passed, 10 total
```

Conclui-se que a refatoração reduziu a complexidade do código sem alterar seu comportamento esperado. O uso dos testes como rede de segurança foi essencial, pois permitiu verificar continuamente se a extração de métodos e a reorganização das responsabilidades mantinham as mesmas saídas para CSV, HTML, administrador, usuário comum e lista vazia. Dessa forma, os testes deram confiança para melhorar o design sem transformar a refatoração em uma mudança arriscada.

A redução de Bad Smells melhora a qualidade geral do software porque torna o código mais legível, facilita a localização de regras de negócio e diminui o esforço necessário para futuras alterações. No caso da classe ReportGenerator, a separação entre filtragem, renderização e cálculo de total deixa o sistema mais preparado para receber novos formatos de relatório ou novas regras de usuário com menos impacto nos comportamentos já existentes.